

Tratamento dos dados

```
''' Importação da biblioteca pandas para ler arquivos CSV em DataFrames
a fim de visualizar seu cabeçalho, verificar sua forma e agrupar dados '''
import pandas as pd

''' Importação da biblioteca seaborn para criar visualizações estatísticas com
gráficos informativos e visualmente atraentes '''
import seaborn as se

''' Importação da biblioteca statistics para fornecer funções a fim de calcular
medidas estatísticas básicas '''
import statistics as st
```

```
# Importa os dados
bank = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/Churn.csv", sep=";")

# Visualiza os dados
bank.head()
```

	X0	X1	X2	X3	X4	X4.1	X6	X7	X8	X9	X10	X11
0	1	619	RS	Feminino	42	2	0	1	1	1	10134888.0	1
1	2	608	SC	Feminino	41	1	8380786	1	0	1	11254258.0	0
2	3	502	RS	Feminino	42	8	1596608	3	1	0	11393157.0	1
3	4	699	RS	Feminino	39	1	0	2	0	0	9382663.0	0
4	5	850	SC	Feminino	43	2	12551082	1	1	1	790841.0	0

```
# Dá nomes às colunas
bank.columns = ["Id", "Score", "Estado", "Genero",
               "Idade", "Patrimonio", "Saldo",
               "Produtos", "TemCartCredito",
               "Ativo", "Salario", "Saiu"]

# Visualiza os dados
bank.head()
```

	Id	Score	Estado	Genero	Idade	Patrimonio	Saldo	Produtos	TemCartCredito	Ativo	Salario	Saiu
0	1	619	RS	Feminino	42	2	0	1	1	1	10134888.0	1
1	2	608	SC	Feminino	41	1	8380786	1	0	1	11254258.0	0
2	3	502	RS	Feminino	42	8	1596608	3	1	0	11393157.0	1
3	4	699	RS	Feminino	39	1	0	2	0	0	9382663.0	0
4	5	850	SC	Feminino	43	2	12551082	1	1	1	790841.0	0

```
# Mostra se existe valores nulos ou não disponíveis (NaN)
bank.isnull().sum()
```

	0
Id	0
Score	0
Estado	0
Genero	8
Idade	0
Patrimonio	0
Saldo	0
Produtos	0
TemCartCredito	0
Ativo	0
Salario	7
Saiu	0

dtype: int64

```
# Agrupa a coluna númerica Salario
bank['Salario'].describe()
```

```

count    9.920000e+02
mean     3.528762e+07
std      5.305800e+08
min      9.677000e+03
25%      3.029011e+06
50%      8.703250e+06
75%      1.405213e+07
max      1.193469e+10
```

```
# Calcula a mediana do salário
mediana = st.median(bank['Salario'])
mediana
```

```
70518.0
```

```
# Substitui os valores nulos pela mediana
'''
fillna: preenche valores ausentes (NaN).
mediana: valor que será usado para preencher os valores ausentes (NaN) na coluna "Salario".
inplace: modificação seja feita diretamente no DataFrame original (bank).
Se inplace=False, o método fillna() retornaria uma cópia do DataFrame com
os valores preenchidos, mas o DataFrame original não seria modificado.
'''
bank['Salario'].fillna(mediana, inplace=True)
```

```

<ipython-input-12-f34dbea28b43>:9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[c

bank['Salario'].fillna(mediana, inplace=True)
```

```
# Mostra se existe valores nulos ou não disponíveis (NaN)
bank.isnull().sum()
```

```

0
Id      0
Score   0
Estado  0
Genero   8
Idade    0
Patrimonio  0
Saldo    0
Produtos  0
TemCartCredito  0
Ativo    0
Salario  0
Saiu     0
```

```
# Mostra se existe valores nulos ou não disponíveis (NAs) na coluna 'Salario'
bank['Salario'].isnull().sum()
```

```
np.int64(0)
```

```
# Mostra se existe valores nulos ou não disponíveis (NAs) na coluna 'Genero'
bank['Genero'].isnull().sum()
```

```
np.int64(8)
```

```
# Preenche as NAs com o valor 'Masculino' (critério: moda)
bank['Genero'].fillna('Masculino', inplace=True)
```

<ipython-input-16-e003d0d06863>:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through `inplace` method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are trying to apply it has been modified by another operation.

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` or `df[col].method(value, inplace=True)`.

```
bank['Genero'].fillna('Masculino', inplace=True)
```

```
# Mostra se existe valores nulos ou não disponíveis (NAs) na coluna 'Genero'
bank['Genero'].isnull().sum()
```

```
np.int64(0)
```

```
# Visualiza a coluna 'Genero' agrupada e sua quantidade
genero = bank.groupby(['Genero']).size()
genero
```

```
0
```

Genero	
F	2
Fem	1
Feminino	461
M	6
Masculino	529

```
'''Para padronizar os valores é preciso usar o método 'loc' do Pandas para
localizar e modificar valores específicos dentro do DataFrame bank.
loc[condição, coluna]
'''
```

```
# Se o valor de 'Genero' for 'M', substituirá na coluna 'Genero' o valor 'Masculino'
bank.loc[bank['Genero'] == 'M', 'Genero'] = 'Masculino'
```

```
# Se o valor de 'Genero' for 'Fem' e 'F', substituirá na coluna 'Genero' o valor 'Feminino'
bank.loc[bank['Genero'].isin(['Fem', 'F']), 'Genero'] = 'Feminino'
```

```
# Visualiza a coluna 'Genero' agrupada e sua quantidade
genero = bank.groupby(['Genero']).size()
genero
```

```
0
```

Genero	
Feminino	464
Masculino	535

```
# Mostra os dados estatísticos da 'Idade'
bank['Idade'].describe()
```



	Idade
count	999.000000
mean	38.902903
std	11.401912
min	-20.000000
25%	32.000000
50%	37.000000
75%	44.000000
max	140.000000

dtype: float64

```
# Localiza os dados de 'Idade' menores que 0 e maiores que 120
bank.loc[(bank['Idade'] <= 0) | (bank['Idade'] > 120)]
```



	Id	Score	Estado	Genero	Idade	Patrimonio	Saldo	Produtos	TemCartCredito	Ativo	Salario	Saiu
851	853	667	PR	Feminino	0	1	9713392	2	0	0	11331677.0	1
867	869	636	RS	Feminino	-10	1	17083346	1	1	0	11051028.0	1
887	889	585	RS	Masculino	0	2	0	2	1	1	1865777.0	0
930	932	661	SC	Masculino	0	6	13262898	1	0	0	3881267.0	0
931	933	564	PR	Masculino	0	9	10352275	2	1	1	3433821.0	0
947	949	715	RS	Feminino	0	5	11859041	1	1	1	568417.0	1
984	986	773	RS	Masculino	-20	1	12453278	2	0	1	1172357.0	0
990	992	655	RS	Masculino	140	5	93147	2	1	0	6621413.0	0

```
# Calcula a mediana
mediana = st.median(bank['Idade'])
mediana
```



37

```
# Substitui os valores de 'Idade' pela mediana
bank.loc[(bank['Idade'] <= 0) | (bank['Idade'] > 120), 'Idade'] = mediana
```

```
# Localiza os dados de 'Idade' menores que 0 e maiores que 120
bank.loc[(bank['Idade'] <= 0) | (bank['Idade'] > 120)]
```



	Id	Score	Estado	Genero	Idade	Patrimonio	Saldo	Produtos	TemCartCredito	Ativo	Salario	Saiu
--	----	-------	--------	--------	-------	------------	-------	----------	----------------	-------	---------	------

```
# Localiza dados duplicados pelo 'Id'
'''
keep=False: especifica que todas as linhas duplicadas
devem ser marcadas como True, o restante será False.
'''
bank.loc[bank.duplicated(['Id'], keep=False)]
```



	Id	Score	Estado	Genero	Idade	Patrimonio	Saldo	Produtos	TemCartCredito	Ativo	Salario	Saiu
80	81	665	RS	Feminino	34	1	9664554	2	0	0	17141366.0	0
81	81	665	RS	Feminino	34	1	9664554	2	0	0	17141366.0	0

```
# Exlui dados duplicados pelo 'Id'
'''
subset='Id': especifica que a coluna Id deve ser usada como referência para identificar as duplicatas.
keep='first': entre as linhas duplicadas, a primeira ocorrência será mantida e as demais serão removidas.
'last': mantém a última ocorrência e remove as demais.
False: remove todas as ocorrências das linhas duplicadas.
inplace=True: Especifica que a modificação (remoção das linhas duplicadas) deve ser feita
diretamente no DataFrame original (bank).
inplace=False (padrão), o método retornaria uma cópia do DataFrame com as linhas duplicadas removidas,
mas o DataFrame original permaneceria inalterado.
'''
bank.drop_duplicates(subset='Id', keep='first', inplace=True)
```

```
# Localiza dados duplicados pelo 'Id'
bank.loc[bank.duplicated(['Id'], keep=False)]
```

```
↗
```

	Id	Score	Estado	Genero	Idade	Patrimonio	Saldo	Produtos	TemCartCredito	Ativo	Salario	Saiu
--	----	-------	--------	--------	-------	------------	-------	----------	----------------	-------	---------	------

```
↖
```

```
# Mostra os estados agrupados
estado = bank.groupby(['Estado']).size()
estado
```

```
↗
```

	0
Estado	
PR	257
RP	1
RS	477
SC	258
SP	4
TD	1

```
↖
```

```
'''
Localiza os estados fora do domínio, no caso ('RP', 'SP', 'TD')
e substitui por um valor específico [critério: moda (no caso, RS)]
'''
```

```
bank.loc[bank['Estado'].isin(['RP', 'SP', 'TD']), 'Estado'] = 'RS'
```

```
# Mostra os estados agrupados
estado = bank.groupby(['Estado']).size()
estado
```

```
↗
```

	0
Estado	
PR	257
RS	483
SC	258

```
↖
```

```
# Calcula o desvio padrão do 'Salario'
desvio = st.stdev(bank['Salario'])
desvio
```

```
↗ 528988918.4679201
```

```
# Localiza salários com o desvio padrão 2x maior
bank.loc[bank['Salario'] >= 2 * desvio]
```

```
↗
```

	Id	Score	Estado	Genero	Idade	Patrimonio	Saldo	Produtos	TemCartCredito	Ativo	Salario	Saiu
7	8	376	PR	Feminino	29	4	11504674	4	1	0	1.193469e+10	1
116	118	668	PR	Feminino	37	6	1678644	1	1	0	1.156383e+10	0
170	172	484	RS	Feminino	29	4	13011439	1	1	0	1.640179e+09	0
230	232	673	RS	Masculino	72	1	0	2	0	1	1.119812e+09	0

```
↖
```

```
# Calcula a mediana de 'Salarios'
mediana = st.median(bank['Salario'])
mediana
```

```
↗ 8637195.5
```

```
# Localiza os salários 2x maior que o desvio padrão e substitui pela mediana
bank.loc[bank['Salario'] >= 2 * desvio, 'Salario'] = mediana
```

```
# Localiza salários com o desvio padrão 2x maior
bank.loc[bank['Salario'] >= 2 * desvio]
```



Id	Score	Estado	Genero	Idade	Patrimonio	Saldo	Produtos	TemCartCredito	Ativo	Salario	Saiu
----	-------	--------	--------	-------	------------	-------	----------	----------------	-------	---------	------

```
# Mostra a quantidade de linhas e colunas do dataset
bank.shape
```



```
(998, 12)
```

```
# Salva o DataFrame em um arquivo CSV
'''
```

```
'produtos.csv' : Nome do arquivo onde o DataFrame será salvo. Pode incluir o caminho, ex.: './dados/meu_arquivo.csv'.
index=False : Não salva o índice do DataFrame como coluna no CSV.
```

```
Geralmente é recomendado para evitar uma coluna extra desnecessária.
```

```
sep=',' : Define o separador de campos. O padrão é ',' (vírgula), mas pode usar outro, como sep=';'.
'''
```

```
bank.to_csv('/content/drive/MyDrive/Colab Notebooks/clientes_banco.csv', index=False)
```